```
```
```
```

```
# Auto-generated by nixos-generate-config
# Edit this configuration file to define what should be installed on
# your system.  Help is available in the configuration.nix(5) man page
# and in the NixOS manual (accessible by running 'nixos-help').

{ config, lib, pkgs, ... }:

{
  imports =
    [ # Include the results of the hardware scan.
      ./hardware-configuration.nix
    ];

  # Use the systemd-boot EFI boot loader.
  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  networking.hostName = "nixos"; # Define your hostname.
  # Pick only one of the below networking options.
  # networking.wireless.enable = true;  # Enables wireless support via wpa_supplicant.
  networking.networkmanager.enable = true;  # Easiest to use and most distros use this by default.

  # Set your time zone.
  time.timeZone = "Europe/Amsterdam";

  # Configure network proxy if necessary
  # networking.proxy.default = "http://user:password@proxy:port/";
  # networking.proxy.noProxy = "127.0.0.1,localhost,internal.domain";

  # Select internationalisation properties.
  i18n.defaultLocale = "en_US.UTF-8";
  console = {
    font = "Lat2-Terminus16";
    keyMap = "us";
    useXkbConfig = true; # use xkbOptions in tty.
  };

  # Enable the X11 windowing system.
  services.xserver.enable = true;


  # Configure keymap in X11
  services.xserver.layout = "us";
  services.xserver.xkbOptions = "eurosign:e";

  # Enable CUPS to print documents.
  services.printing.enable = true;

  # Enable sound.
  sound.enable = true;
  hardware.pulseaudio.enable = true;

  # Enable touchpad support (enabled default in most desktopManager).
  # services.xserver.libinput.enable = true;

  # Some programs need SUID wrappers, can be configured further or are
  # started in user sessions.
  # programs.mtr.enable = true;
  # programs.gnupg.agent = {
  #   enable = true;
  #   enableSSHSupport = true;
  # };

  # List services that you want to enable:

  # Enable the OpenSSH daemon.
  # services.openssh.enable = true;

  # Open ports in the firewall.
  # networking.firewall.allowedTCPPorts = [ ... ];
  # networking.firewall.allowedUDPPorts = [ ... ];
  # Or disable the firewall altogether.
  # networking.firewall.enable = false;

  # This value determines the NixOS release from which the default
  # settings for stateful data, like file locations and database versions
  # on your system were taken. It's perfectly fine and recommended to leave
  # this value at the release version of the first install of this system.
  # Before changing this value read the documentation for this option
  # (e.g. man configuration.nix or on https://nixos.org/nixos/options.html).
  system.stateVersion = "20.09"; # Did you read the comment?

}
```

```
# config_manager.py
import os
import json
import threading
from typing import Any, Dict, Optional

class ConfigManager:
    """Thread-safe configuration manager with environment variable overrides."""

    def __init__(self, config_path: Optional[str] = None, env_prefix: str = "APP_"):
        self._config: Dict[str, Any] = {}
        self._lock = threading.RLock()
        self._env_prefix = env_prefix
        if config_path:
            self.load_file(config_path)

    def load_file(self, path: str) -> None:
        with open(path, "r") as f:
            data = json.load(f)
        with self._lock:
            self._config.update(data)

    def get(self, key: str, default: Any = None) -> Any:
        env_key = self._env_prefix + key.upper().replace(".", "_")
        if env_key in os.environ:
            return os.environ[env_key]
        with self._lock:
            return self._config.get(key, default)

    def set(self, key: str, value: Any) -> None:
        with self._lock:
            self._config[key] = value

    def __repr__(self) -> str:
        with self._lock:
            return f"ConfigManager(keys={list(self._config.keys())})"
```

```
# test_config_manager.py
import os
import pytest
from config_manager import ConfigManager

def test_get_default():
    cfg = ConfigManager()
    assert cfg.get("missing", "fallback") == "fallback"

def test_set_and_get():
    cfg = ConfigManager()
    cfg.set("db.host", "localhost")
    assert cfg.get("db.host") == "localhost"

def test_env_override(monkeypatch):
    cfg = ConfigManager()
    cfg.set("db.host", "localhost")
    monkeypatch.setenv("APP_DB_HOST", "prod-db")
    assert cfg.get("db.host") == "prod-db"

def test_load_file(tmp_path):
    p = tmp_path / "cfg.json"
    p.write_text('{"a": 1, "b": 2}')
    cfg = ConfigManager(str(p))
    assert cfg.get("a") == 1
    assert cfg.get("b") == 2
```

```
# cli.py
import argparse
import sys
from config_manager import ConfigManager

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="cfg", description="Config CLI")
    p.add_argument("-f", "--file", help="config file path")
    p.add_argument("command", choices=["get", "set", "dump"])
    p.add_argument("key", nargs="?")
    p.add_argument("value", nargs="?")
    return p

def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    cfg = ConfigManager(args.file)
    if args.command == "get":
        print(cfg.get(args.key))
    elif args.command == "set":
        cfg.set(args.key, args.value)
    elif args.command == "dump":
        print(cfg)
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

```
# Makefile
.PHONY: test lint run

test:
    pytest -q

lint:
    ruff check .

run:
    python cli.py dump
```

```
# .github/workflows/ci.yml
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install pytest ruff
      - run: make lint
      - run: make test
```

```
# pyproject.toml
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.pytest.ini_options]
testpaths = ["."]
```

```
# README.md
# config_manager

Thread-safe configuration manager with environment variable overrides.

## Usage

```
cfg -f config.json get db.host
cfg set db.host localhost
cfg dump
```

Development

```
make test
make lint
```
```

```
# LICENSE
MIT License

Copyright (c) 2026 Example Author

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

```
```

```
## 8.10 snapshot-aware log

Snapshots let us discard the prefix of the log that the state machine has already absorbed. The log then starts at some index greater than 1, and every piece of code that assumed "index 1 is the first entry" has to be taught otherwise.

### 8.10.1 Log compaction

Compaction is triggered by the application, not by Raft. The state machine knows when it has applied enough entries that a snapshot is worthwhile, so it asks Raft to take one:

```
func (rf *Raft) Snapshot(index int, snapshot []byte) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    if index <= rf.lastIncludedIndex {
        return // already have a snapshot at least this new
    }
    rf.lastIncludedTerm = rf.termAt(index)
    rf.log = append([]LogEntry(nil), rf.log[index-rf.lastIncludedIndex:]...)
    rf.lastIncludedIndex = index
    rf.persistWithSnapshot(snapshot)
}
```

8.10.2 InstallSnapshot RPC

When a follower is so far behind that the leader has already discarded the entries it needs, the leader sends the whole snapshot in one RPC:

```
type InstallSnapshotArgs struct {
    Term              int
    LeaderId          int
    LastIncludedIndex int
    LastIncludedTerm  int
    Data              []byte
}

type InstallSnapshotReply struct {
    Term int
}

func (rf *Raft) sendInstallSnapshot(server int) {
    rf.mu.Lock()
    args := InstallSnapshotArgs{
        Term:              rf.currentTerm,
        LeaderId:          rf.me,
        LastIncludedIndex: rf.lastIncludedIndex,
        LastIncludedTerm:  rf.lastIncludedTerm,
        Data:              rf.persister.ReadSnapshot(),
    }
    rf.mu.Unlock()
    var reply InstallSnapshotReply
    if rf.peers[server].Call("Raft.InstallSnapshot", &args, &reply) {
        rf.mu.Lock()
        defer rf.mu.Unlock()
        if reply.Term > rf.currentTerm {
            rf.becomeFollower(reply.Term)
            return
        }
        rf.matchIndex[server] = args.LastIncludedIndex
        rf.nextIndex[server] = args.LastIncludedIndex + 1
    }
}
```

8.10.3 Persisting snapshots

The snapshot and the log must be persisted together, otherwise a crash between the two writes can leave them inconsistent:

```
func (rf *Raft) persistWithSnapshot(snapshot []byte) {
    w := new(bytes.Buffer)
    e := labgob.NewEncoder(w)
    e.Encode(rf.currentTerm)
    e.Encode(rf.votedFor)
    e.Encode(rf.lastIncludedIndex)
    e.Encode(rf.lastIncludedTerm)
    e.Encode(rf.log)
    rf.persister.SaveStateAndSnapshot(w.Bytes(), snapshot)
}
```

8.10.4 Applying snapshots

On the follower side, the RPC handler installs the snapshot and tells the state machine to reset:

```
func (rf *Raft) InstallSnapshot(args *InstallSnapshotArgs, reply *InstallSnapshotReply) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    reply.Term = rf.currentTerm
    if args.Term < rf.currentTerm {
        return
    }
    if args.Term > rf.currentTerm {
        rf.becomeFollower(args.Term)
    }
    rf.resetElectionTimer()
    if args.LastIncludedIndex <= rf.lastIncludedIndex {
        return
    }
    rf.log = []LogEntry{{Term: args.LastIncludedTerm}}
    rf.lastIncludedIndex = args.LastIncludedIndex
    rf.lastIncludedTerm = args.LastIncludedTerm
    rf.persistWithSnapshot(args.Data)
    rf.applyCh <- ApplyMsg{SnapshotValid: true, Snapshot: args.Data,
        SnapshotTerm: args.LastIncludedTerm, SnapshotIndex: args.LastIncludedIndex}
}
```

8.11 membership changes

Changing the set of servers in a running cluster is dangerous: if the old and new configurations can each form a majority on their own, two leaders can be elected in the same term. Raft avoids this by going through a joint configuration.

8.11.1 Joint consensus

During the transition, every decision must be approved by a majority of the old configuration *and* a majority of the new one:

```
type Config struct {
    Old []int
    New []int // empty when not in joint consensus
}

func (c Config) quorum(votes map[int]bool) bool {
    if !majority(c.Old, votes) {
        return false
    }
    if len(c.New) > 0 && !majority(c.New, votes) {
        return false
    }
    return true
}
```

8.11.2 Single-server changes

The simpler alternative is to add or remove one server at a time. Any two majorities of configurations that differ by one server must overlap, so the cluster can never split:

```
func (rf *Raft) addServer(id int) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    if rf.state != Leader {
        return
    }
    rf.config.New = append(rf.config.Old, id)
    rf.nextIndex[id] = rf.lastLogIndex() + 1
    rf.matchIndex[id] = 0
    rf.appendEntry(LogEntry{Term: rf.currentTerm, Config: rf.config})
}
```

8.11.3 Learners

A new server should not vote until it has caught up, or it will slow every commit. Learners receive replication but are excluded from quorum:

```
func (rf *Raft) isVoter(id int) bool {
    for _, v := range rf.config.Old {
        if v == id {
            return true
        }
    }
    for _, v := range rf.config.New {
        if v == id {
            return true
        }
    }
    return false
}
```

8.12 client interaction

Clients talk to the cluster through a small RPC interface. They do not know who the leader is, so they guess and follow redirects.

8.12.1 Finding the leader

The client remembers the last server that answered successfully and tries it first:

```
type Clerk struct {
    servers []*labrpc.ClientEnd
    leader  int
    id      int64
    seq     int
}

func (ck *Clerk) Get(key string) string {
    for {
        var reply GetReply
        ok := ck.servers[ck.leader].Call("KV.Get", &GetArgs{Key: key}, &reply)
        if ok && reply.Err == OK {
            return reply.Value
        }
        ck.leader = (ck.leader + 1) % len(ck.servers)
    }
}
```

8.12.2 Duplicate detection

Retries can cause the same command to be applied twice. Each client tags its requests with a unique id and a sequence number, and the state machine remembers the last sequence it executed per client:

```
type KV struct {
    mu      sync.Mutex
    data    map[string]string
    lastSeq map[int64]int
}

func (kv *KV) apply(op Op) {
    if op.Seq <= kv.lastSeq[op.ClientId] {
        return // duplicate
    }
    kv.lastSeq[op.ClientId] = op.Seq
    switch op.Type {
    case "Put":
        kv.data[op.Key] = op.Value
    case "Append":
        kv.data[op.Key] += op.Value
    }
}
```

8.12.3 Read-only optimization

Reads can skip the log entirely if the leader first confirms it still has a quorum, using a heartbeat round:

```
func (rf *Raft) readIndex() int {
    rf.mu.Lock()
    if rf.state != Leader {
        rf.mu.Unlock()
        return -1
    }
    idx := rf.commitIndex
    rf.mu.Unlock()
    rf.broadcastHeartbeat()
    return idx
}
```

8.13 testing

Distributed bugs are timing bugs, so the test harness matters as much as the code. The lab provides a network that can drop, reorder, and delay messages.

8.13.1 Unreliable network

Tests run with a configurable failure rate and long delays, which surfaces races that a clean network would hide:

```
func (net *Network) setReliable(yes bool) {
    net.mu.Lock()
    defer net.mu.Unlock()
    net.reliable = yes
    if yes {
        net.longDelays = false
    }
}
```

8.13.2 Partitions

A partition is simulated by disabling the endpoints between two groups of servers:

```
func partition(net *Network, a, b []int) {
    for _, i := range a {
        for _, j := range b {
            net.Enable(endname(i, j), false)
            net.Enable(endname(j, i), false)
        }
    }
}
```

8.13.3 Checking linearizability

Each client records the invocation and response of every operation, and a checker searches for a sequential history that explains them:

```
type Op struct {
    Input  interface{}
    Call   int64
    Output interface{}
    Return int64
}

func checkLinearizable(ops []Op, model Model) bool {
    return search(ops, model, 0, make(map[uint64]bool))
}
```

8.14 performance

Once correctness is settled, the remaining work is to make the common case fast: batching, pipelining, and avoiding unnecessary disk writes.

8.14.1 Batching

Instead of one RPC per entry, the leader accumulates entries for a short window and sends them together:

```
func (rf *Raft) batcher() {
    for {
        time.Sleep(10 * time.Millisecond)
        rf.mu.Lock()
        if rf.state == Leader && len(rf.pending) > 0 {
            rf.log = append(rf.log, rf.pending...)
            rf.pending = nil
            rf.persist()
        }
        rf.mu.Unlock()
    }
}
```

8.14.2 Pipelining

The leader does not wait for one AppendEntries to finish before sending the next; it tracks an optimistic `nextIndex` and corrects it when replies arrive out of order.

8.14.3 Persistence cost

`fsync` dominates latency. Group commit amortizes it across many entries, and writing the log to a dedicated device avoids seek contention with the state machine.

8.15 summary

Implementing Raft is mostly about getting the boring details right: persist before you reply, never trust a stale term, and assume every RPC can be lost, duplicated, or delayed. The algorithm itself is small; the test suite is where the time goes.
```

```
configuration_manager_main
    create_router_maintenance_configuration_main_with_timeout_default
    initialize_database_main_pool_connection_main_max_connections_default
    create_main_configuration_file_if_not_exists_main
    load_main_configuration_from_file_main
    run_main
    log_info_maintenance_router_ok
    shutdown_main_ok_fn
    initialize_main_ok_fn
    initialize_main_ok_fn_fn
    initialize_main_ok_fn_fn_fn
    initialize_main_ok_fn_fn_fn_fn
    run_main_ok_fn
    shutdown_main_ok_fn_fn
    shutdown_main_ok_fn_fn_fn
    shutdown_main_ok_fn_fn_fn_fn
    start_router
    start_router_fn
    start_router_fn_fn
    start_router_fn_fn_fn
    start_router_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn_fn
    start_router_fn
```

```
# Generated by Django 4.2.11 on 2024-04-17 19:39

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("auth", "0012_alter_user_first_name_max_length"),
    ]

    operations = [
        migrations.CreateModel(
            name="Group",
            fields=[
                ("id", models.AutoField(auto_created=True, primary_key=True, serialize=False, verbose_name="ID")),
                ("name", models.CharField(max_length=150, unique=True, verbose_name="name")),
                ("permissions", models.ManyToManyField(blocked=False, to="auth.permission", verbose_name="permissions")),
            ],
            options={
                "verbose_name": "group",
                "verbose_name_plural": "groups",
            },
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-04-17 19:39

import django.contrib.auth.models
import django.contrib.auth.validators
from django.db import migrations, models
from django.utils import timezone


class Migration(migrations.Migration):

    initial = True

    dependencies = [
        ("auth", "0001_initial"),
    ]

    operations = [
        migrations.CreateModel(
            name="User",
            fields=[
                ("id", models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name="ID")),
                ("password", models.CharField(max_length=128, verbose_name="password")),
                ("last_login", models.DateTimeField(blocked=False, null=True, verbose_name="last login")),
                (
                    "is_superuser",
                    models.BooleanField(
                        default=False,
                        help_text="Designates that this user has all permissions without explicitly assigning them.",
                        verbose_name="superuser status",
                    ),
                ),
                (
                    "username",
                    models.CharField(
                        error_messages={"unique": "A user with that username already exists."},
                        help_text="Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.",
                        max_length=150,
                        unique=True,
                        validators=[django.contrib.auth.validators.UnicodeUsernameValidator()],
                        verbose_name="username",
                    ),
                ),
                ("first_name", models.CharField(blank=True, max_length=150, verbose_name="first name")),
                ("last_name", models.CharField(blank=True, max_length=150, verbose_name="last name")),
                ("email", models.EmailField(blank=True, max_length=254, verbose_name="email address")),
                (
                    "is_staff",
                    models.BooleanField(
                        default=False,
                        help_text="Designates whether the user can log into this admin site.",
                        verbose_name="staff status",
                    ),
                ),
                (
                    "is_active",
                    models.BooleanField(
                        default=True,
                        help_text="Designates whether this user should be treated as active. Unselect this instead of deleting accounts.",
                        verbose_name="active",
                    ),
                ),
                ("date_joined", models.DateTimeField(default=timezone.now, verbose_name="date joined")),
                (
                    "groups",
                    models.ManyToManyField(
                        blocked=False,
                        help_text="The groups this user belongs to. A user will get all permissions granted to each of their groups.",
                        related_name="user_set",
                        related_query_name="user",
                        to="auth.group",
                        verbose_name="groups",
                    ),
                ),
                (
                    "user_permissions",
                    models.ManyToManyField(
                        blocked=False,
                        help_text="Specific permissions for this user.",
                        related_name="user_set",
                        related_query_name="user",
                        to="auth.permission",
                        verbose_name="user permissions",
                    ),
                ),
            ],
            options={
                "verbose_name": "user",
                "verbose_name_plural": "users",
                "abstract": False,
            },
            managers=[
                ("objects", django.contrib.auth.models.UserManager()),
            ],
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-04-17 19:39

import django.contrib.auth.models
import django.contrib.auth.validators
from django.db import migrations, models
from django.utils import timezone


class Migration(migrations.Migration):

    initial = True

    dependencies = [
        ("auth", "0001_initial"),
    ]

    operations = [
        migrations.CreateModel(
            name="User",
            fields=[
                ("id", models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name="ID")),
                ("password", models.CharField(max_length=128, verbose_name="password")),
                ("last_login", models.DateTimeField(blocked=False, null=True, verbose_name="last login")),
                (
                    "is_superuser",
                    models.BooleanField(
                        default=False,
                        help_text="Designates that this user has all permissions without explicitly assigning them.",
                        verbose_name="superuser status",
                    ),
                ),
                (
                    "username",
                    models.CharField(
                        error_messages={"unique": "A user with that username already exists."},
                        help_text="Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.",
                        max_length=150,
                        unique=True,
                        validators=[django.contrib.auth.validators.UnicodeUsernameValidator()],
                        verbose_name="username",
                    ),
                ),
                ("first_name", models.CharField(blank=True, max_length=150, verbose_name="first name")),
                ("last_name", models.CharField(blank=True, max_length=150, verbose_name="last name")),
                ("email", models.EmailField(blank=True, max_length=254, verbose_name="email address")),
                (
                    "is_staff",
                    models.BooleanField(
                        default=False,
                        help_text="Designates whether the user can log into this admin site.",
                        verbose_name="staff status",
                    ),
                ),
                (
                    "is_active",
                    models.BooleanField(
                        default=True,
                        help_text="Designates whether this user should be treated as active. Unselect this instead of deleting accounts.",
                        verbose_name="active",
                    ),
                ),
                ("date_joined", models.DateTimeField(default=timezone.now, verbose_name="date joined")),
                (
                    "groups",
                    models.ManyToManyField(
                        blocked=False,
                        help_text="The groups this user belongs to. A user will get all permissions granted to each of their groups.",
                        related_name="user_set",
                        related_query_name="user",
                        to="auth.group",
                        verbose_name="groups",
                    ),
                ),
                (
                    "user_permissions",
                    models.ManyToManyField(
                        blocked=False,
                        help_text="Specific permissions for this user.",
                        related_name="user_set",
                        related_query_name="user",
                        to="auth.permission",
                        verbose_name="user permissions",
                    ),
                ),
            ],
            options={
                "verbose_name": "user",
                "verbose_name_plural": "users",
                "abstract": False,
            },
            managers=[
                ("objects", django.contrib.auth.models.UserManager()),
            ],
        ),
    ]
```

```
```

```
sql_user
```

```
    func
    if err != nil {
        return nil, fmt.Errorf("error: %w", err)
    }
    return result, nil
}

// Config represents application configuration
type Config struct {
    Host     string `yaml:"host"`
    Port     int    `yaml:"port"`
    Database string `yaml:"database"`
    Debug    bool   `yaml:"debug"`
}

func loadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }
    var cfg Config
    if err := yaml.Unmarshal(data, &cfg); err != nil {
        return nil, err
    }
    return &cfg, nil
}

func main() {
    // Load configuration from file
    cfg, err := loadConfig("config.yaml")
    if err != nil {
        log.Fatalf("failed to load config: %v", err)
    }
    fmt.Printf("Starting server on %s:%d\n", cfg.Host, cfg.Port)
    // Initialize database connection
    db, err := sql.Open("postgres", fmt.Sprintf("host=%s port=%d dbname=%s", cfg.Host, cfg.Port, cfg.Database))
    if err != nil {
        log.Fatalf("failed to connect to database: %v", err)
    }
    defer db.Close()
    // Create HTTP server with timeouts
    server := &http.Server{
        Addr:         fmt.Sprintf(":%d", cfg.Port),
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        IdleTimeout:  120 * time.Second,
    }
    log.Fatal(server.ListenAndServe())
}

# Advanced Python decorators and metaclasses
import functools
import time
import logging
from typing import Any, Callable, Dict, List, Optional, Type, TypeVar, Generic
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from contextlib import contextmanager
from collections import defaultdict, OrderedDict
from weakref import WeakValueDictionary
logger = logging.getLogger(__name__)
T = TypeVar('T')
F = TypeVar('F', bound=Callable[..., Any])
def retry(max_attempts: int = 3, delay: float = 1.0, backoff: float = 2.0,
          exceptions: tuple = (Exception,)) -> Callable[[F], F]:
    """Decorator that retries a function with exponential backoff."""
    def decorator(func: F) -> F:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            current_delay = delay
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    logger.warning(
                        f"Attempt {attempt}/{max_attempts} failed for "
                        f"{func.__name__}: {e}"
                    )
                    if attempt < max_attempts:
                        time.sleep(current_delay)
                        current_delay *= backoff
            raise last_exception
        return wrapper
    return decorator
def memoize(maxsize: int = 128):
    """Simple memoization decorator with LRU eviction."""
    def decorator(func):
        cache = OrderedDict()
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            key = (args, tuple(sorted(kwargs.items())))
            if key in cache:
                cache.move_to_end(key)
                return cache[key]
            result = func(*args, **kwargs)
            cache[key] = result
            if len(cache) > maxsize:
                cache.popitem(last=False)
            return result
        wrapper.cache = cache
        wrapper.cache_clear = lambda: cache.clear()
        return wrapper
    return decorator
class RegistryMeta(type):
    """Metaclass that automatically registers subclasses."""
    _registry: Dict[str, Type] = {}
    def __new__(mcs, name, bases, namespace):
        cls = super().__new__(mcs, name, bases, namespace)
        if bases:  # Don't register the base class itself
            mcs._registry[name] = cls
        return cls
    @classmethod
    def get_registry(mcs) -> Dict[str, Type]:
        return dict(mcs._registry)
class Plugin(metaclass=RegistryMeta):
    """Base class for plugins."""
    @abstractmethod
    def execute(self, data: Any) -> Any:
        pass
    @abstractmethod
    def validate(self, data: Any) -> bool:
        pass
class JSONPlugin(Plugin):
    def execute(self, data: Any) -> Any:
        import json
        return json.dumps(data, indent=2)
    def validate(self, data: Any) -> bool:
        try:
            import json
            json.dumps(data)
            return True
        except (TypeError, ValueError):
            return False
@dataclass
class Event:
    type: str
    payload: Dict[str, Any] = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    source: Optional[str] = None
class EventEmitter:
    def __init__(self):
        self._listeners: Dict[str, List[Callable]] = defaultdict(list)
        self._once_listeners: Dict[str, List[Callable]] = defaultdict(list)
    def on(self, event_type: str, callback: Callable) -> None:
        self._listeners[event_type].append(callback)
    def once(self, event_type: str, callback: Callable) -> None:
        self._once_listeners[event_type].append(callback)
    def off(self, event_type: str, callback: Callable) -> None:
        self._listeners[event_type] = [
            cb for cb in self._listeners[event_type] if cb != callback
        ]
    def emit(self, event: Event) -> None:
        for callback in self._listeners.get(event.type, []):
            callback(event)
        for callback in self._once_listeners.pop(event.type, []):
            callback(event)
@contextmanager
def timer(label: str = ""):
    """Context manager that measures execution time."""
    start = time.perf_counter()
    yield
    elapsed = time.perf_counter() - start
    print(f"{label}: {elapsed:.4f}s")
if __name__ == "__main__":
    @retry(max_attempts=3, delay=0.5)
    @memoize(maxsize=64)
    def fetch_data(url: str) -> dict:
        import urllib.request
        with urllib.request.urlopen(url) as resp:
            return {"status": resp.status, "length": len(resp.read())}
    with timer("fetch"):
        print(fetch_data("https://example.com"))
    print("Registered plugins:", RegistryMeta.get_registry())

// Event loop and task scheduler
#include <iostream>
#include <queue>
#include <functional>
#include <chrono>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <atomic>
#include <vector>
#include <memory>
#include <unordered_map>
#include <string>
#include <sstream>
#include <iomanip>
#include <ctime>
#include <cstdlib>
#include <cstring>
#include <cassert>
#include <stdexcept>
#include <algorithm>
#include <numeric>
#include <fstream>
#include <filesystem>
#include <optional>
#include <variant>
#include <any>
#include <tuple>
#include <array>
#include <set>
#include <map>
#include <deque>
#include <list>
#include <stack>
#include <bitset>
#include <random>
#include <regex>
#include <iterator>
#include <limits>
#include <type_traits>
#include <utility>
#include 
```